

CCF-GESP C++四级

编程能力等级认证



第六课

结 构 体

01 结构体的定义



知识讲解

结构体

结构体是由多个类型不同（或相同）的数据组合而成的**数据类型**。

成员变量

```
struct 结构体名{  
    数据类型1 变量名1;  
    数据类型2 变量名2;  
    .....  
};
```

```
struct stu{  
    int id;  
    string name;  
    double score;  
};
```

```
stu a={1,"yaya",98};  
a.score=95;  
cout<<a.score<<endl;
```



知识讲解

结构体变量所占内存



结构体变量所占**内存**，是**所有成员变量**所需内存空间的**总和**

```
struct st{
    char c;
    int id;
    double s;
}a={'a',1,80};
cout<<sizeof(a.c)<<endl;
cout<<sizeof(a.id)<<endl;
cout<<sizeof(a.s)<<endl;
cout<<sizeof(a)<<endl;
```

1+4+8→13

1
4
8
16

为什么计算
a的存储空间
是16?



知识讲解

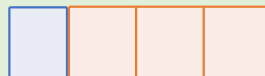
对齐补齐原则



结构体变量的存储空间为**最宽成员变量存储单元的整数倍**，若所有成员变量相加计算出来的总空间**不是**其整数倍，则**补齐**为它的整数倍。

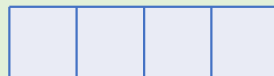
```
struct st{  
    char c;  
    int id;  
    double s;  
} a = {'a', 1, 80};
```

c(char)

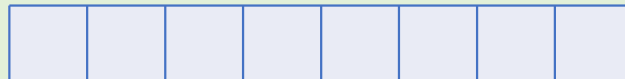


对齐补齐

id(int)



s(double) 最宽



a

$4 + 4 + 8 \rightarrow 16$



知识讲解

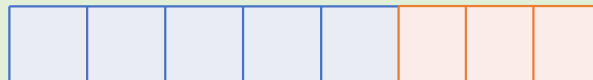
对齐补齐原则



结构体变量的存储空间为**最宽成员变量存储单元的整数倍**，若所有成员变量相加计算出来的总空间**不是**其整数倍，则**补齐**为它的整数倍。

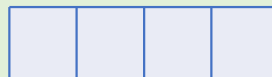
```
struct sd{  
    char c[5];  
    int i;  
}b={"1234",100};
```

c(char)



对齐补齐

i(int) 最宽



8+4→12

b



真题解析

判断题

(2023年9月)在C++语言中，可以通过定义结构体，定义一个新的数据类型。

☒ 正确

错误 ☐



课堂练习

选择题

设有以下说明语句，则下面的叙述**不正确**的是 （ C ）。

```
struct stu{  
    int a;  
    float b;  
} stutype;
```

- A. struct 是结构体类型的关键字
- B. struct stu 是用户定义的结构体类型
- C. stutype 是用户定义的结构体类型
- D. a 和 b 都是结构体成员名



课堂练习

选择题

当说明一个结构体变量时，系统分配给它的内存是（ **A** ）

- A. 各成员所需内存量的总和，并遵守对齐补齐原则
- B. 结构中第一个成员所需内存量
- C. 成员中占内存量最大者所需的容量
- D. 结构中最后一个成员所需内存量



课堂练习

选择题

结构体类型变量在程序执行期间 (**A**)

- A. 所有成员一直驻留在内存中
- B. 只有一个成员驻留在内存中
- C. 部分成员驻留在内存中
- D. 没有成员驻留在内存中



知识讲解

结构体嵌套结构体



结构体内的**成员变量**是另一个**结构体**

```
struct student  
{  
    string name;  
    int age;  
    int score;  
};  
struct teacher  
{  
    int id;  
    string name;  
    int age;  
    struct student stu;  
};
```

```
teacher t = {100, "大白", 45};
```

```
t.stu.name = "小童";
```

```
t.stu.age = 12;
```

```
t.stu.score = 100;
```

```
cout<<t.name<<" "<<t.id<<" "<<t.age<<endl;
```

```
cout<<t.stu.name<<" ";
```

```
cout<<t.stu.age<<" "<<t.stu.score;
```

```
大白 100 45  
小童 12 100
```



知识讲解

结构体作为函数参数

```
struct student
{
    string name;
    int age;
    int score;
};
void stuInfo(student s) {
    cout<<s.name<<endl;
    s.name = "小白";
}
```

```
student s={"小童",12,100};
stuInfo(s);
cout<<s.name;
```

小童
小童

值传递是把**实参**赋值给形参。



知识讲解

结构体作为函数参数

```
struct student
{
    string name;
    int age;
    int score;
};
void stuInfo(student *s) {
    cout<<s->name<<endl;
    s->name = "小白";
}
```

```
student s={"小童", 12, 100};
student *p = &s;
stuInfo(p);
cout<<s.name;
```

小童
小白

地址传递对形参和实参指向**同一个**结构体变量！



知识讲解

结构体中const的使用



const作用：防止对结构体进行**误操作**！

```
struct student
{
    string name;
    int age;
    int score;
};
void stuInfo(const student *s){
    cout<<s->name<<endl;
    s->name = "小白";
}
```

不允许修改，程序报错

```
student s={"小童", 12, 100};
student *p = &s;
stuInfo(p);
cout<<s.name;
```

02 结构体数组



知识讲解

定义结构体数组

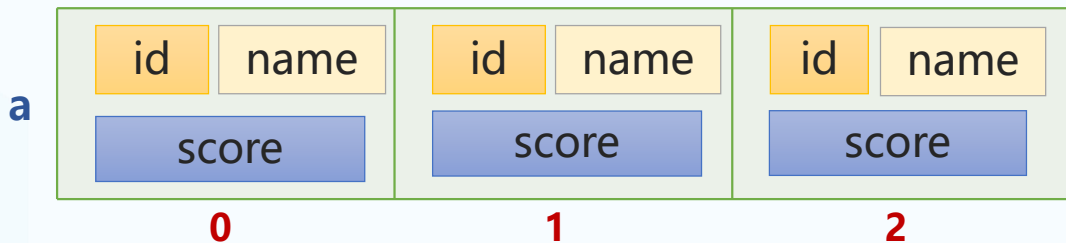


像使用int类型一样，使用结构体类型定义数组。

```
struct stu{  
    int id;  
    char name[15];  
    double score;  
};  
stu a[3];
```

```
struct stu{  
    int id;  
    char name[15];  
    double score;  
}a[3];
```

定义结构体的
同时定义数组





知识讲解

结构体数组的初始化



定义数组的**同时**，直接初始化

```
struct stu {  
    int id;  
    char name[15];  
    double score ;  
};  
stu a[3]={1,"yaya",98,2,"qiqi",82.5,3,"lili",89};
```

a[0]

a[1]

a[2]

a	1	yaya	2	qiqi	3	lili
	98	82.5	89			
	0		1		2	



知识讲解

结构体数组的初始化



定义数组的**同时**，使用{}进行初始化

推荐

```
struct stu {  
    int id;  
    char name[15];  
    double score ;  
};  
stu a[3]={ {1, "yaya", 98}, {2, "qiqi", 82.5}, {3, "lili", 89} };
```

a[0]

a[1]

a[2]

a	1	yaya	2	qiqi	3	lili
	98	82.5	89			
	0		1		2	



知识讲解

数组元素的成员变量



访问成员变量语法: **数组名[下标].成员变量**

```
struct stu {  
    int id;  
    char name[15];  
    double score ;  
};  
stu a[2]={{1, "yaya", 98},{2, "qiqi", 82.5},{3, "lili", 89}};  
  
cout << a[0].id << endl;  
cout << a[0].name << endl;  
cout << a[0].score << endl;
```

Diagram illustrating array access:

- `a[0].name` points to the `name` member of the first element.
- `a[0].id` points to the `id` member of the first element.
- `a[0].score` points to the `score` member of the first element.

Output of the code:

```
1  
yaya  
98
```



知识讲解

数组元素的输入

输入:

```
1 yaya 98
2 qiqi 82.5
3 lili 89
```

```
struct stu{
    int id;
    char name[15];
    double score;
}a[3];

int main(){
    for(int i=0;i<3;i++)
        cin>>a[i].id>>a[i].name>>a[i].score;

    for(int i=0;i<3;i++)
        cout<<a[i].id<<" "<<a[i].name<<" "<<a[i].score<<endl;

    return 0;
}
```

定义结构体的同时
创建数组



知识讲解

结构体数组排序



对结构体数组**按分数升序**排序，**必须**自定义排序规则

id	name	id	name	id	name
1	yaya	2	qiqi	3	lili
98		82.5		89	
score		score		score	

升序排序规则

```
bool cmp( stu x, stu y ) {  
    return x.score < y.score;  
}
```

升序用小于号

sort(a, a+3, cmp);



真题解析

判断题

(2023年9月)在C++语言中，可以定义结构体类型的数组变量，定义结构体时也可以包含数组成员。

☒ 正确

错误 ☐



课堂练习

选择题

下面程序运行的结果是 (D)

```
struct cmplx {  
    int x;  
    int y;  
} cnum[2]={1,3,2,7};  
  
printf("%d\n",cnum[0].y/cnum[0].x*cnum[1].x);
```

A. 0

B. 1

C. 3

D. 6



课堂练习

选择题

根据下面的定义，能打印出字母 M 的语句是 (D)

```
struct person {  
    char name[9];  
    int age;  
}  
person a[10] = {  
    "John", 17,  
    "Paul", 19,  
    "Mary", 18,  
    "adam", 16  
};
```

- A. printf ("%c\n" ,a[3].name);
- B. printf ("%c\n" ,a[3].name [1]);
- C. printf ("%c\n" ,a[2].name [1]);
- D. printf ("%c\n" ,a[2].name [0]);



课堂练习

通话次数

【问题描述】

手机是最常用的通信工具，童童和好朋友之间经常打电话相互问候，现在输入一次通话中两位朋友的姓名，请编程统计一下通话次数最多的人的通话次数。

【输入描述】

共 $n+1$ 行；

第一行，一个正整数 n ，表示有 n 对朋友通过话， $1 \leq n \leq 1000$ ；

接下来 n 行，每行都有两个用空格隔开的姓名，表示通过电话的一对朋友。每个人的姓名仅由小写英文字母组成，不含空格，且 $1 \leq \text{姓名长度} \leq 15$ 。

【输出描述】

一行，一个字符串和一个整数，分别表示通话次数最多的人姓名，和通话次数，两个数据之间使用空格隔开，保证通话次数最多的人只有一个。

【输入样例】

```
4
tongtong lili
meimei tongtong
lili meimei
tongtong yaya
```

【输出样例】

```
tongtong 3
```



课堂练习

思路分析

统计每个人的
通话次数



找到通话次数
最多的人



输出通话次数做
多的人名和对应
的通话次数



课堂练习

统计每个人的通话次数

- ➡ tongtong lili
- ➡ meimei tongtong
- ➡ lili meimei
- ➡ tongtong yaya

姓名: tongtong	姓名: lili	姓名: meimei	姓名: yaya
通话次数: 3	通话次数: 2	通话次数: 2	通话次数: 1

- 将**n对**好友的通话信息记录在一个数组中
- 当前通话人在数组中**没有记录**，就新添加该通话人，通话次数记为**1**
- 如果当前通话人数组中**已有记录**，则通话次数**加1**



课堂练习

实现步骤

- ① 定义结构体类型 con，用于记录通话人的姓名和通话次数



con

```
struct con{  
    string name;  
    int cnt;  
};
```

姓名

通话次数



课堂练习

实现步骤

- ① 定义结构体类型 con，用于记录通话人的姓名和通话次数
- ② 创建结构体数组a，用于保存n对通话人的信息



姓名
通话次数

con

```
struct con{  
    string name;  
    int cnt;  
}a[2010];
```

数组长度，
最大n*2



课堂练习

实现步骤

- ③ 输入n和n对好友通话信息。
- ④ 一边输入一边统计通话次数cnt
 - 创建**bool**变量，用于标记数组中是否已有通话人记录。

false 没有 **true** 有

```
int n;  
cin >> n;  
string x;  
for(int i=0; i<n*2; i++){  
    cin >> x;  
    bool f=false;  
    for(int j=0; j<=i; j++){  
        if(a[j].name==x){  
            f=true;  
            a[j].cnt++;  
            break;}  
    }  
}
```

循环n*2次

标记通话人是否已存在

已有，通话次数++



课堂练习

实现步骤

- ③ 输入n和n对好友通话信息。
- ④ 一边输入一边统计通话次数cnt
 - 创建**bool**变量，用于标记数组中是否已有通话人记录。

false 没有 **true** 有

```
int n;  
cin >> n;  
string x;  
for(int i=0; i<n*2; i++){  
    cin >> x;  
    bool f=false;  
    for(int j=0; j<=i; j++){  
        if(a[j].name==x){  
            f=true;  
            a[j].cnt++;  
            break;}  
    }  
    if(!f){  
        a[i].name=x;  
        a[i].cnt=1;  
    }  
}
```

没有，新添加到数组中



课堂练习

实现步骤

⑤ 找出最多通话次数的人，按要求输出

```
int max=0;
string maxn="";
for(int i=0;i<n*2;i++){
    if(max<a[i].cnt){
        max=a[i].cnt;
        maxn=a[i].name;
    }
}
cout<<maxn<<" "<<max;
```



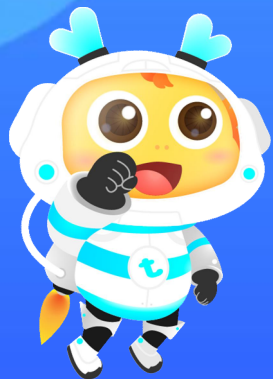
课堂练习

【完整代码】

```
#include<iostream>
using namespace std;
struct con{
    string name;
    int cnt;
}a[2010];
int main(){
    int n;
    cin>>n;
```

```
    string x;
    for(int i=0;i<n*2;i++){
        cin>>x;
        bool f=false;
        for(int j=0;j<=i;j++){
            if(a[j].name==x){
                f=true;
                a[j].cnt++;
                break;
            }
        }
        if(!f){
            a[i].name=x;
            a[i].cnt=1;
        }
    }
}
```

```
int max=0;
string maxn="";
for(int i=0;i<n*2;i++){
    if(max<a[i].cnt){
        max=a[i].cnt;
        maxn=a[i].name;
    }
}
cout<<maxn<<" "<<max;
return 0;
```



感谢你的学习